



*Division of Computing Science and Mathematics
Faculty of Natural Sciences
University of Stirling*

**Design and Development of a Fintech Mobile Application
Using React Native and TrueLayer Open Banking APIs**

Fuad Oluwatosin Laguda

**Dissertation submitted in partial fulfilment for the degree of
Master of Science in Financial Technology**

September 2025

Abstract

The technological boom of digital banking services has created a new reality where users must manage multiple financial institution applications to have complete knowledge of their finances. The European Union's Payment Services Directive 2 (PSD2) was designed to create third-party access to banking data and payment services, and it was this directive that opened the door for Open Banking. Open Banking platforms like TrueLayer's are there to make it easy to access banking services from their APIs. This should make the development of third-party apps easier as developers can focus on integrating only the Open Banking platform, and not on integrating multiple financial institutions APIs into the app. However, the practical implementation of this Open Banking platform reveals that there are gaps between the theoretical regulatory promises and development realities.

This project developed a React Native 0.72.6 mobile banking application integrating TrueLayer's Open Banking sandbox APIs, Google Firebase authentication, and Google Gemini AI services. Using an iterative Agile development approach the application consolidates multiple bank accounts into a unified dashboard. The application also implements secure authentication protocols and has an AI-powered transaction categorisation which uses a hybrid keyword-LLM approach. The application functions within a sandbox environment using mock banking data provided by TrueLayer's Open Banking platform and user experience evaluation was conducted through hands-on testing, following Nielsen's usability testing methodology.

The application successfully achieved core Open Banking integration objectives with 99% API connection success rate and an authentication performance of less than 2 seconds. User feedback showed that the unified dashboard concept improved financial oversight through consolidated account views. However, there were challenges in the development process that required architectural pivots, and one of these was caused by SDK compatibility that resulted in using a WebView approach for the payment section. These challenges revealed the un-addressed gaps between Open Banking's theories and practical development realities that current literature does not cover. The project exposes the hidden costs in development time and infrastructure dependencies developers have to consider before they commence on such mobile application development.

Attestation

I understand the nature of plagiarism, and I am aware of the University's academic integrity policy.

I certify that this dissertation reports original work by me during my MSc project except for the following:

- The development methodology described in Section 3.2 was based on Agile principles as referenced in the literature [2].
- The TrueLayer Open Banking APIs and authentication flows discussed in Section 3.3 and 3.5.1 were provided by TrueLayer and used in accordance with their developer documentation and terms of service.
- The Firebase Authentication and Firestore database services discussed in Section 3.3 and 3.4.4 were provided by Google and used in accordance with their terms of service.
- The Google Gemini AI service for transaction categorisation discussed in Section 3.4.3 and 3.5.3 was provided by Google and used in accordance with their API terms of service.
- The server-side cryptographic signing solution discussed in Section 3.5.1 was deployed using Render cloud hosting service in accordance with their terms of service.
- The React Native framework and third-party libraries listed in the project dependencies (including react-native-keychain, react-native-webview, axios, crypto-js, and others) were used in accordance with their respective open-source licenses.

Signature

Fuad Oluwatosin Laguda

Date August 19th, 2025

Acknowledgements

I am grateful to my supervisor, Dr. Mario Kolberg, whose guidance and support made this project experience rewarding. To Seun Omosowon, who provided advice and practical support, and to Akpo Ekpeki, whose guidance on project structure and supervisor relationships helped in shaping my approach. I am also grateful to Emmanuel Oladoyin, who always encouraged me to remain focused.

I would also like to appreciate my aunts and my sister who provided emotional and mental support throughout this journey, and my parents.

Consent for the University to Share your Dissertation

Please sign your name underneath one of the options below, to indicate whether you consent to the University sharing your dissertation (e.g. as an example to future students):

The University may make this dissertation, with indicative grade, available to others.

The University may make this dissertation available to others, but the grade may not be disclosed.

A handwritten signature in blue ink, appearing to read 'Fuad', with a stylized flourish extending from the end.

Fuad Oluwatosin Laguda

August 19th, 2025

The University may not make this dissertation available to others.

Table of Contents

Abstract	i
Attestation.....	ii
Acknowledgements	iii
Consent for the University to Share your Dissertation	iv
Table of Contents.....	v
List of Figures.....	vii
1 Introduction	1
1.1 Background and Context	1
1.2 Research Questions.....	1
1.3 Scope and Objectives	1
1.4 Main Contribution.....	2
1.5 Overview of Dissertation.....	2
2 State-of-The Art	3
2.1 Introduction.....	3
2.2 Open Banking Evolution and Regulatory Framework	3
2.2.1 Open Banking Transformation Theory	3
2.2.2 Implementation Gap Between Theory and Practice	3
2.3 Security Architectures in Mobile Banking	4
2.3.1 Authentication Paradigms and Their Limitations.....	4
2.3.2 Cryptographic Requirements and Implementation Theory	4
2.4 Human-Computer Interaction in Financial Contexts.....	5
2.5 AI-Powered Transaction Categorisation: Hybrid Approach.....	5
3 Methodology and Implementation.....	6
3.1 Introduction.....	6
3.2 Development Methodology	6
3.3 Technology Stack and Rationale	6
3.4 Implementation Process.....	7
3.4.1 System Architecture Overview.....	7
3.4.2 Initial Setup and Configuration	8
3.4.3 Core Feature Development.....	9
3.4.4 Security Implementation.....	14
3.4.5 Testing and Quality Assurance	15
3.5 Challenges and Resolutions.....	15
3.5.1 Payment Authorisation Challenge.....	16
3.5.2 Production Deployment and Device Testing	17
3.5.3 Slow Loading of Spending Category and AI API cost.....	17
3.6 Lessons Learned and Summary.....	17
4 Evaluation.....	18
4.1 Introduction.....	18
4.2 User Experience Evaluation.....	18
4.2.1 Navigation and Usability Assessment	18
4.2.2 Performance and Reliability Perception.....	19
4.2.3 Security Perception and Trust	19
4.3 Technical Performance Assessment	19
4.3.1 TrueLayer Open Banking API Integration Effectiveness	19
4.3.2 Authentication Performance.....	20
4.3.3 Unified Dashboard Performance.....	21

4.4	Compliance and Security Assessment.....	21
4.5	Evaluation Summary.....	21
5	Conclusion.....	23
5.1	Introduction.....	23
5.2	Comparison to Other Work	23
5.3	Limitations	23
5.4	Critical Implementation Challenges and Practical Insights	24
5.5	Industry Implications and Practical Insights.....	24
5.6	Future Work	24
5.7	Conclusion	25
	References.....	26

List of Figures

Figure 3.1. System Architecture Overview	7
Figure 3.2. Android Build Configuration Modifications	8
Figure 3.3. Gradle Properties Modifications	8
Figure 3.4. Deep Linking Configuration	8
Figure 3.5. File Structure	9
Figure 3.6. Cache and Sync Service Implementation	10
Figure 3.7. Unified Dashboard Screen.....	10
Figure 3.8. Transaction History Screen	11
Figure 3.9. TrueLayer Service Implementation	11
Figure 3.10. Payment Screen Initiation	12
Figure 3.11. Payment Screen.....	12
Figure 3.12. Spending Categorisation	13
Figure 3.13. Spending Categorisation Screen	13
Figure 3.14. Category Management Screen.....	14
Figure 3.15. Payment Flow	16

1 Introduction

1.1 Background and Context

The technological boom of digital banking services has created a new reality where users must manage multiple financial institution applications to have complete knowledge of their finances.

The European Union's Payment Services Directive 2 (PSD2) was designed to create third-party access to banking data and payment services, and it was this directive that opened the door for Open Banking. Open Banking platforms like TrueLayer's are there to make it easy to access banking services from their APIs. This should make the development of third-party apps easier as developers can focus on integrating only the Open Banking platform, and not on integrating multiple financial institutions APIs into the app. However, the practical implementation of this Open Banking platform reveals that there are gaps between the theoretical regulatory promises and development realities.

This project looks into these development realities by developing a React Native mobile application and integrating it to TrueLayer's Open Banking APIs. The developed application demonstrates what is achievable, the constraints and gaps in the current Open Banking infrastructure.

The application consolidates multiple bank accounts into a unified dashboard, implements secure authentication protocols, and has an AI-powered transaction categorisation. The application functions within a sandbox environment using mock banking data provided by TrueLayer's Open Banking platform.

The development process exposed the differences between existing literature on Open Banking integration and the practical challenges developers face. Practical challenges faced in this development are, SDK compatibility, dependency management, and application performance that current literature do not address.

1.2 Research Questions

The research questions this dissertation aims to address are:

1. How effectively can TrueLayer's Open Banking APIs be integrated into a React Native application?
2. How can a unified financial dashboard improve users' financial management experience?
3. What gaps exist between Open Banking's theoretical frameworks and practical development realities?

These questions come from the observation that while Open Banking efforts promise simplified third-party integration, the practical implementation experiences reveal unaddressed challenges.

1.3 Scope and Objectives

The project's scope is to develop a working mobile banking application using React Native 0.72.6. The application will be integrated with Google Firebase, TrueLayer's Open Banking sandbox APIs, and Google Gemini AI API. The objectives are:

- Implement dual user authentication: email/password and biometric verification.
- Integrate TrueLayer's Open Banking APIs for account aggregation and payment services.
- Create a unified dashboard to display the authenticated user's consolidated account information.

- Develop an AI-powered transaction categorisation using a hybrid keyword and LLM approach.
- Conduct user experience testing and evaluate user responses on the application's performance.
- Document the implementation challenges and architectural decisions used to achieve the application's functionality of the above objectives.

1.4 Main Contribution

This work makes three contributions to the understanding of Open Banking implementation. First, it provides documented evidence of the technical challenges that developers may face when integrating third-party applications with Open Banking platforms. This documentation fills the gaps in existing literature that focuses primarily on theoretical benefits, and not the technical challenges and performance trade-offs developers must navigate. Second, it shares functional integration solutions to challenges that occurred during development, which may be useful to developers attempting similar implementations. Third, it provides performance and user experience data that shows the capabilities and constraints of current Open Banking approaches.

1.5 Overview of Dissertation

Chapter 2 looks at the existing literature on banking regulations, Open Banking, security architectures, and AI-powered financial services. The chapter identifies the gaps that exist between theoretical frameworks and practical implementation requirements.

Chapter 3 documents the methodology and implementation process used in developing the application, including the technical pivots used when the primary approaches proved incompatible with development constraints.

Chapter 4 evaluates the application's performance based on user feedback experience. It also critically evaluates the application performance, and compliance requirements, assessing both the achievements and limitations of the application.

Chapter 5 presents critical implementation insights and recommends directions for future work that scales the project beyond the sandbox development to a production environment. It also shares suggestions on how to improve user experience and improve the integration of Open Banking platforms with third-party applications.

2 State-of-The Art

2.1 Introduction

The digital transformation of mobile financial services through Open Banking has created room for extensive research, and this literature review covers theoretical foundations and challenges in the implementation of Open Banking platforms.

The focus of this chapter will be on API integration, data security, and user experience design, and by reviewing existing knowledge on these three aspects, this chapter will evaluate how they apply to the current technological environment.

Although Open Banking has made data portability possible and opened the door for third-party access, the practical implementation of secure, user-friendly mobile applications remains complex and under-researched.

2.2 Open Banking Evolution and Regulatory Framework

2.2.1 Open Banking Transformation Theory

Zachariadis and Ozcan [1] theoretical framework on Open Banking transformation, was based on a case study of European banks done over several years. In this study, for Open Banking to be successful, there must be a balance between innovation, security, and regulatory compliance. Their analysis focused only on traditional banks and not third-party providers (TPPs). Also, their work assumes the presence of a solid API infrastructure, not the reality of sandbox/testing environments limitations that may differ when deploying the application in a production environment.

The European regulatory framework came up with the PSD2 [18], which is supposed to help create a structured environment for Open Banking implementation. However, the regulation's documented technical standards have a lot of ambiguities that leaves room for different interpretations and causes implementation inconsistencies as documented in the European Commission's study [19].

2.2.2 Implementation Gap Between Theory and Practice

Early PSD2 implementation showed that there was a disconnect between regulatory intention and technical reality, because while PSD2 mandated standardised APIs, many financial institutions implemented their own proprietary APIs. This has caused an ongoing issue between Third Party Providers (TPPs) and banks because of implementation inconsistencies in PSD2 API deployment [20].

Currently, things have changed and there is substantial progress toward harmonisation, with the Berlin Group's NextGenPSD2 framework being implemented by more than 75% of European banks [37][38].

The initial challenge of standardisation banks faced was often as a result of development limitations, security concerns, or competitive advantages, requiring technological differentiation from basic standard requirements [21].

Even though there is more than 75% adoption by European banks, there are still variations when it comes to API implementations, but current literature suggests these are actually driven by legitimate business needs and not regulatory non-compliance [39].

A new directive is being worked on, it will be PSD3, it is meant to address many of the other challenges holding back standardisation by eliminating fallback options that allowed proprietary implementations [40]. This upcoming directive shows that regulatory evolution are bridging gaps through improved frameworks and clearer technical standards.

Looking at the standardisation challenges, another way to close the gap will be the adoption of Open Banking, by doing this TPPs only have to focus on implementing the Open Banking Platform API.

2.3 Security Architectures in Mobile Banking

2.3.1 Authentication Paradigms and Their Limitations

Most mobile banking security frameworks reference OWASP's Mobile Security Testing Guide [22] but applying the guide in Open Banking application development has revealed implementation challenges. OWASP's guide requires some adaptation when using it with mobile banking application security requirements such as fraud, customer authentication, and API interoperability banking institutions face [42].

The OWASP Mobile Top 10 2024 puts improper credential usage and weak authentication/authorisation as major risks in mobile application development, which are relevant to consider in Open Banking implementations [41]. This creates implementation gaps where Open Banking implementation must balance between current mobile security guides and practical regulatory compliance.

An ideal Open Banking security should implement dual user authentication mechanisms, email and password and biometric authentication. This is supported by the proposed PSD3's Strong Customer Authentication (SCA) guidelines, which supports biometric and multi-factor authentication to strengthen mobile app security for banking applications [43], [44].

2.3.2 Cryptographic Requirements and Implementation Theory

The implementation of payment services in Open Banking platforms require specific cryptographic standards as documented in section 3.5.1. An example of an Open Banking Platform payment cryptography is ECDSA ES512 digital signature algorithm which uses P-521 elliptic curve and SHA-512 hashing, this algorithm is used by TrueLayer [10]. This cryptography example complies with NIST SP 800-186 recommendations for elliptic curve cryptography and FIPS 186-5 digital signature requirements [29], [30].

The reason for such high-level cryptography is to make sure that payment request is authentic and non-repudiable. The P-521 curve offers 256-bit security strength, which protects the request from current cryptographic attacks [29] and the SHA-512 hashing is for integrity verification of the payment request [31].

For cryptographic signing, OWASP guidelines [11] recommend server-side cryptographic operations, however the choice between client-side signing and server-side signing is not addressed in the OWASP guidelines. Industry standard, NIST SP 800-57 discusses considerations for key management and the impact of key location on security [23].

Which brings forth the question, in payment cryptography should it be client-side or server-side?

Client-side cryptography allows for user control over private keys and requires severe computational resources which can affect application performance [13]. It also removes the reliance of signing from a single source, which if attacked, the whole infrastructure is compromised. However, it increases the likely source of potential attack as the banks do not have control on the user's habit, or the user devices storage or security management [11]. On the other hand, server-side signing offers a superior performance when it comes to the computational resource. It also limits the source of failure from every banking user who downloaded the app, to a single source server. This allows for more focused security control as recommended by the NIST [23].

Both methods have their own security advantages and disadvantages, but when it comes to performance and user-experience in application development, server-side signing will be more appropriate.

2.4 Human-Computer Interaction in Financial Contexts

Nielsen's study [9], which found that five users can identify 85% of usability issues has been widely adopted as it simplifies the evaluation process. However, to capture the full range of problems including functionality and security, more extensive testing is required which five user's feedback cannot capture. Nielsen prioritises efficiency over thoroughness, but in the financial industry where errors can have legal, reputational and financial impact on the institution, the benefit of expanding the test users outweighs the cost.

Faulkner's study [17] supports this point, showing that five-user groups detected between 55% and nearly 99% of issues, but groups of ten were consistently able to detect at least 80%. The gap between the lower bound of Nielsen and Faulkner is 25%, and that 25% may be the flaw that can cause a financial institution compliance violation or its licence.

For this project, five users will be enough to deliver the balance between evaluation, time availability, and financial resources needed to complete this project.

2.5 AI-Powered Transaction Categorisation: Hybrid Approach

Transaction categorisation in financial applications often benefit from a hybrid approach [26] instead of a direct Large Language Model (LLM) approach to sorting. This hybrid model uses keyword sorting first to sort the data, then uses LLM to sort the unknowns, this balances efficiency with adaptability [26]. Another way to sort will be to use the LLMs only, this can be useful when dealing with a small quantity of data, but in large quantity, like a user's full transactional history, it may be slow. The slowness of the LLMs comes from computational overhead when dealing with large volume of data [32]. Y. Nie et al., study shows that in processing financial documents like transactional list, there is inference costs involved when using LLMs [36].

In both cases, there is a security risk issue of exposure during data transmission, storage, and processing [33]. A financial institution may want to resolve this by creating their own in-house LLM to protect users' data from being sent to a third-party LLMs to sort [27]. With the model in-house, the institution can guarantee security of user data and not violate any regulation. However, there is the maintenance cost of hosting, storing and processing users' data which can grow exponentially with usage [34].

To resolve both the security and cost limitations, modern third-party LLM providers like Microsoft Azure offer enterprise-grade security frameworks which are compliant with financial regulations such as GDPR and Basel III [35]. Also, hosting with third-party LLMs comes with the added benefit of model growth, which comes at no expense to the institution. LLMs trained on diverse datasets will perform better than models trained only on smaller institution dataset [28], third-party LLM providers can leverage a broader dataset to improve performance.

From the above the advantage of having AI needs met by a third-party LLM whose security infrastructure and policies meets the regulated standard, outweighs trying to build an in-house model.

3 Methodology and Implementation

3.1 Introduction

This chapter covers the development of a mobile banking application, and it documents the implementation steps used to connect to TrueLayer's Open Banking APIs sandbox, from a React Native 0.72.6 environment. The project's main aim was to address the issue of having multiple banking applications. This implementation follows the Open Banking principles by Zachariadis and Ozcan [1] and lets the user access their multiple bank accounts from within a single app, while maintaining a secure authentication process through Google Firebase.

During the development process, multiple technical challenges were faced, which then required architectural changes to be able to complete the project but also stick to the objectives. This chapter documents every technical challenge, the solutions applied, and implementation decisions. It also includes code examples from the actual project implementation.

3.2 Development Methodology

The project required a flexible approach to development to navigate technical roadblocks, while being focused on the main objectives. An iterative method based on Agile principles was used to aid in the navigation of the various challenges during development [2]. This Agile iterative method was chosen over others like waterfall development, because it allowed the testing of each iterative cycle independently due to unknown technical challenges expected when working with APIs and React Native dependencies.

The application development was grouped into these phases: authentication, API integration and user interface design. This made it easy to handle build issues and debugging needed to set up a solid foundation before adding more layers of functionality.

These functionalities were implemented in weekly iterative cycles so as to allow enough time to implement, test and resolve technical issues before moving to the next phase. Longer periods would mean investing too much time on an implementation, instead of looking for an alternative when faced with a roadblock, and shorter periods would mean not allowing enough time for testing. The iterative approach helped guard against frequent restarts when faced with build challenges, as it was easy to roll back to what worked and start again from there, rather than to build the entire environment from scratch.

3.3 Technology Stack and Rationale

React Native was selected over other native Android and iOS development because, it has cross-platform compatibility making implementation easy. Other native development would have required the setting up of different environments for both Android and iOS.

Evaluation of native versus cross-platform showed that React Native achieves near native performance compared to separate native applications [14], [15], this aligns with the evaluation shared by Rieger and Majchrzak [16]. This near native performance in financial applications can be the difference in achieving a proper integration of cryptographic operations and secure storage implementation.

React Native 0.72.6 was chosen because creating a working stable build environment with versions 0.79.2, 0.73.4, and 0.70.6 was not possible after multiple attempts to resolve the compatibility issues. This was due to issues like Kotlin version mismatch and build.gradle configurations which further compounded the SDK challenges and required other dependency changes. These mismatches caused build failures, and too much time was invested in trying to resolve these, which delayed project completion. React Native 0.72.6 did not have these mismatches, and it allowed the development to focus on stability and security, a practical approach for application development instead of access to new features [3].

For the purpose of this project, speed and simplicity were directives as the app is not meant to be an enterprise level application but should be able to scale to enterprise level. This is why Render cloud hosting, Google Gemini AI and Firebase Authentication were chosen due to their ease of integration and free usage capacity. Also, Firebase comes with database (Firestore) and free hosting without any other third-party integration.

However, in a production environment paid alternatives like AWS Cognito would be more appropriate because of auditability and advanced access controls.

3.4 Implementation Process

3.4.1 System Architecture Overview

The system architecture in figure 3.1 shows the four-layer design used to make clear separation of concerns and have a maintainable code structure with security implementation functioning across all layers.

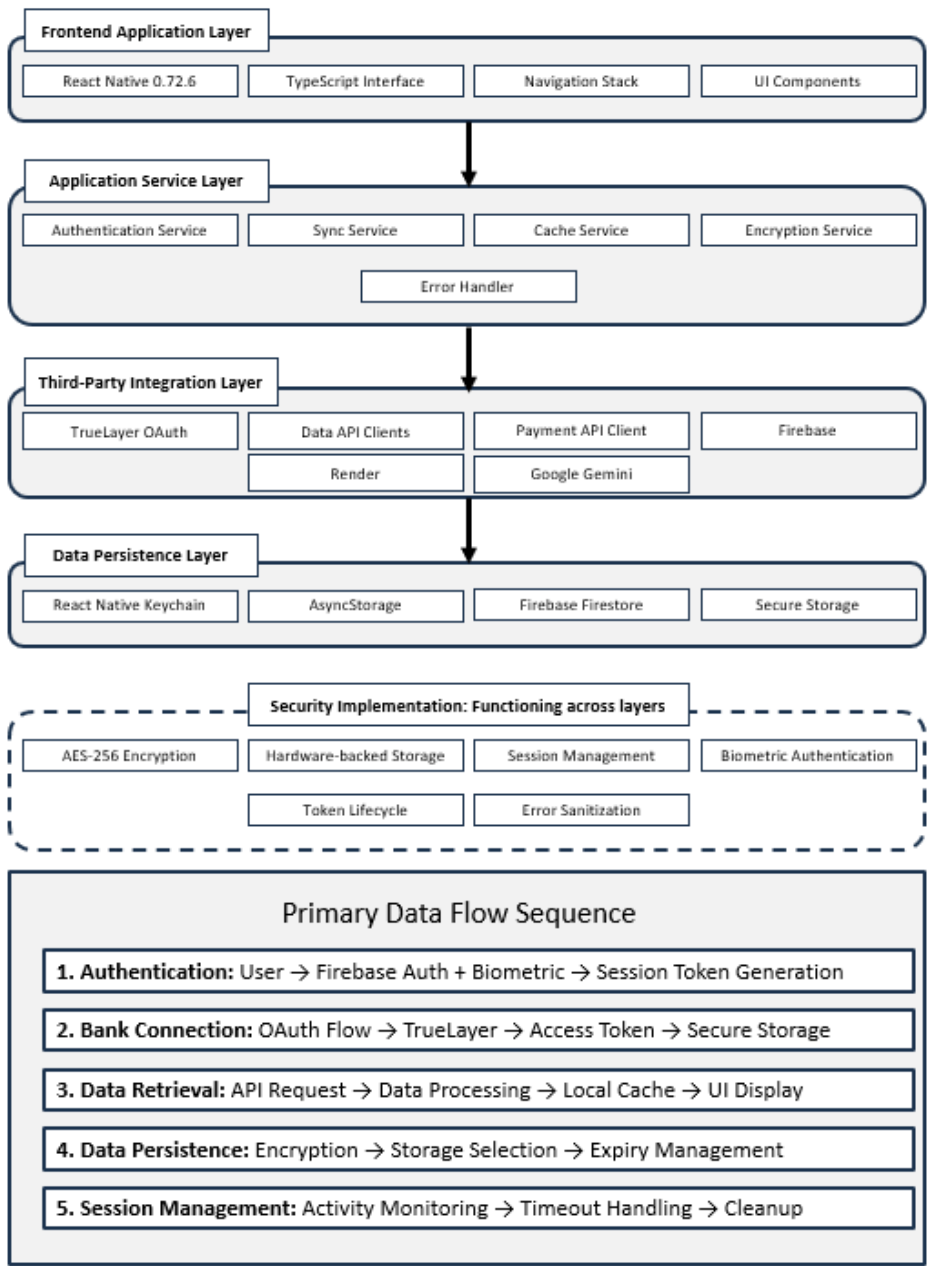


Figure 3.1. System Architecture Overview

3.4.2 Initial Setup and Configuration

The development began with configuring the React Native environment, with the focus being on creating an app that will be compatible with various Android versions.

Figure 3.2 below is of the android/build.gradle file, which required some changes to its default settings by adding the Firebase dependencies, and changing the build:gradle, to 7.4.2. Flipper debugging tool plugin was disabled in android/app/build.gradle, as it caused long build time of around 12 minutes, after being disabled build time dropped to 5 minutes, improving development and testing.

```
}  
repositories {  
    google()  
    mavenCentral()  
}  
dependencies {  
    classpath("com.android.tools.build:gradle:7.4.2")  
    classpath("com.facebook.react:react-native-gradle-plugin")  
    classpath("com.google.gms:google-services:4.3.15")  
}
```

Figure 3.2. Android Build Configuration Modifications

Figure 3.3 shows the gradle.properties modifications needed to resolve the issue regarding memory management in the build process. This low memory was causing "Java heap space" errors that prevented successful builds, this required increasing the JVM memory allocation in gradle.properties.

```
# gradle.properties  
org.gradle.jvmargs=-Xmx2048m -XX:MaxMetaspaceSize=512m  
android.useAndroidX=true  
android.enableJetifier=true  
hermesEnabled=true
```

Figure 3.3: Gradle Properties Modifications

Figure 3.4 shows the added deep linking configuration implemented in the AndroidManifest.xml, to enable TrueLayer authentication callbacks and support the return flow from the API processes.

```
<intent-filter>  
    <action android:name="android.intent.action.VIEW" />  
    <category android:name="android.intent.category.DEFAULT" />  
    <category android:name="android.intent.category.BROWSABLE" />  
    <data android:scheme="truelayerbankingapp" android:host="callback" />  
</intent-filter>
```

Figure 3.4: Deep Linking Configuration

3.4.3 Core Feature Development

The project was developed using modular architecture with separate directories for components, screens, services, and utilities, as seen in figure 3.5 to help with the testing and debugging of each section. This architecture helped to isolate different components of the project, and modifications made to one aspect, would not break the functionality of other aspects.

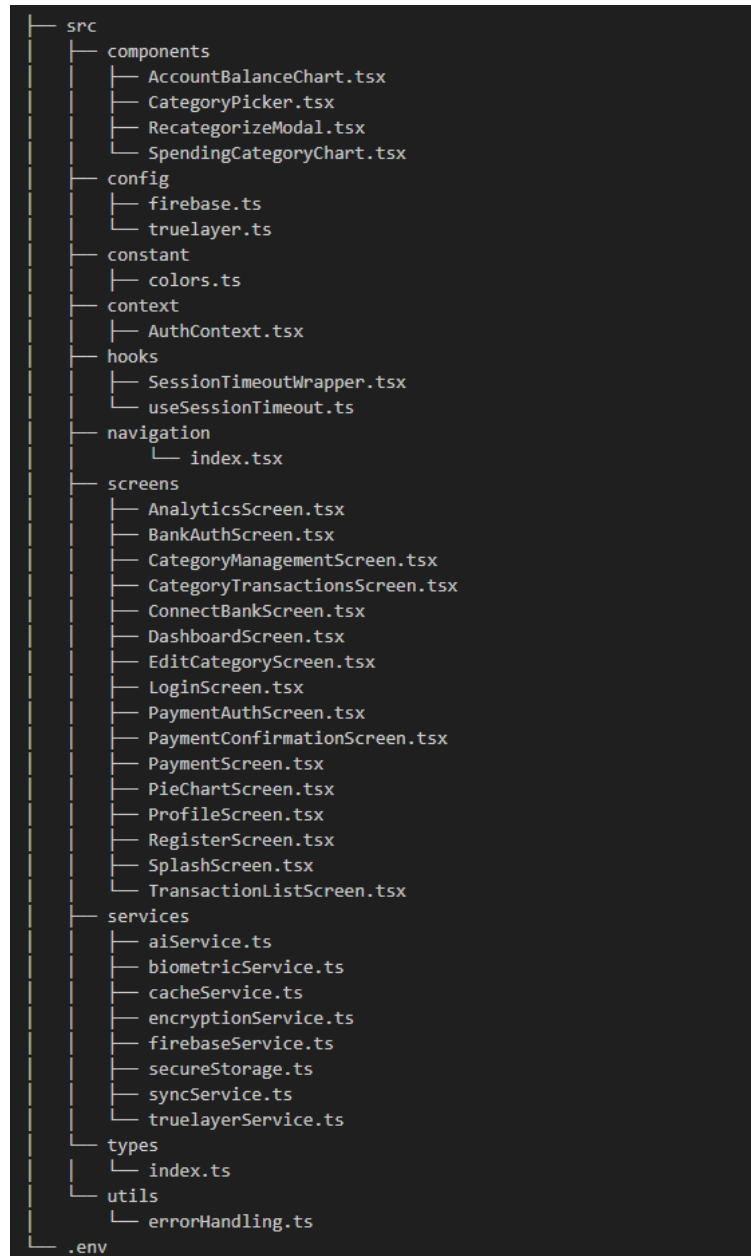


Figure 3.5: File Structure

Bank Account Fetching

Figure 3.6 shows how the app used TrueLayer's APIs to fetch and display account balances and transactions, and the data fetched was then cached locally using both the `cacheService.ts` and `SyncService.ts` for a period of 2 minutes.

```

// src/services/cacheService.ts
async getCache<T>(key: string, options: CacheOptions = {}): Promise<T | null> {
  const serialized = await AsyncStorage.getItem(`cache_${key}`);
  if (!serialized) return null;
  const cachedItem: CachedItem<T> = JSON.parse(serialized);
  if (options.expiryMinutes && now - cachedItem.timestamp > expiryMs) {

// src/services/syncService.ts
async getAccounts(): Promise<(BankAccount & { balance: Balance | null })[]> {
  const cachedAccounts = await cacheService.getCache('accounts', {
    expiryMinutes: 10
  });
  if (cachedAccounts && cachedAccounts.length > 0) {
    return cachedAccounts;
  }
}

```

Figure 3.6: Cache and Sync Service Implementation

Figure 3.7 shows the unified dashboard after the bank accounts has been fetched and aggregated.

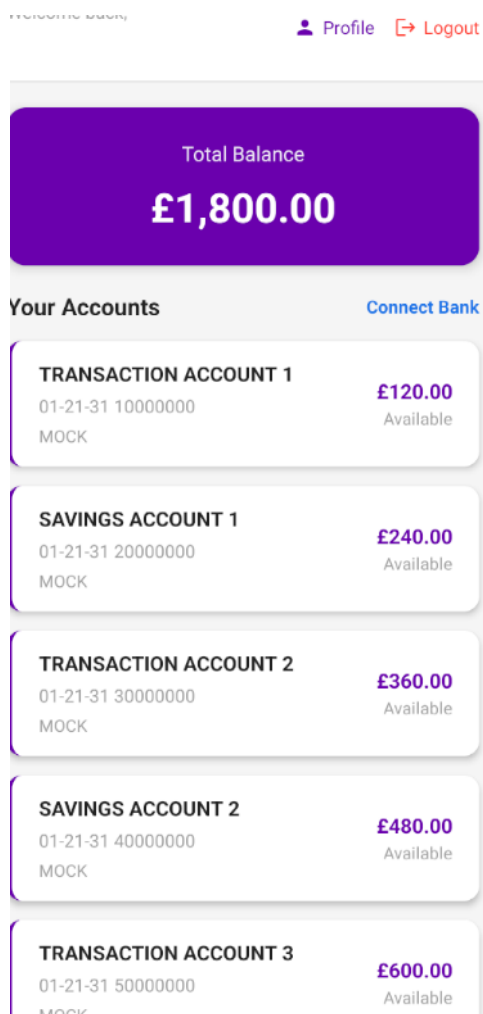


Figure 3.7: Unified Dashboard Screen

Figure 3.8 shows all the transaction history of the fetched bank accounts.

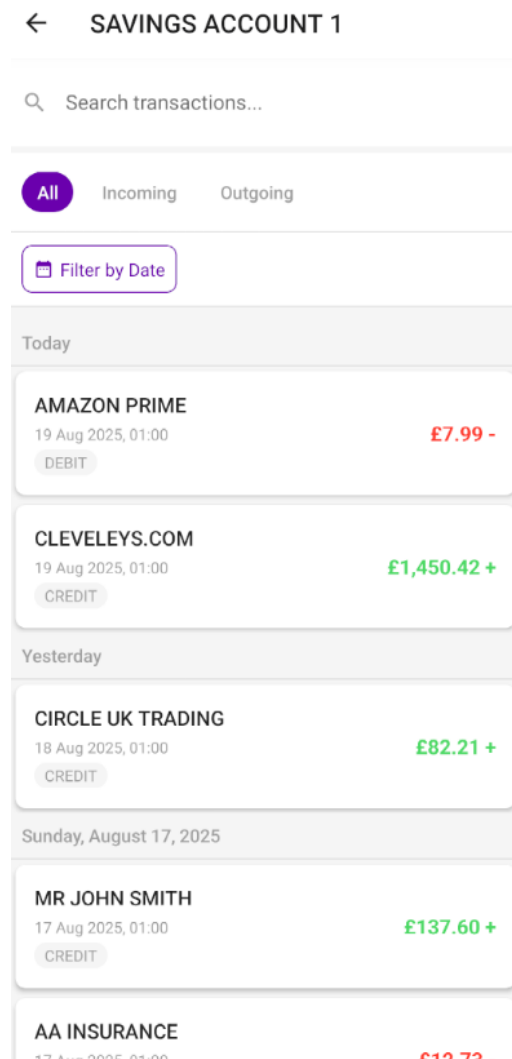


Figure 3.8: Transaction History Screen

Figure 3.9 is a sample code of the trueLayerService.ts and how it handles API integration, error handling and secure token management for safe and reliable data retrieval.

```
// src/services/trueLayerService.ts
async authenticatedFetch(url: string, options: any = {}): Promise<any> {
  const token = await this.getAccessToken();
  if (!token) {
    throw new Error('No access token available. Please connect a bank first.');
```

Figure 3.9: TrueLayer Service Implementation

Payment Initiation

Figure 3.10 is a sample code of the PaymentScreen.tsx, and how it works within TrueLayer's sandbox environment. The payment system was designed to be user-friendly and trustworthy,

through easy navigation, well-labelled fields, and real-time validation [4]. The design has input validations to guide the user through each step, while a dedicated confirmation screen showed the payment status updates, so there is transparency throughout the whole process.

```
// src/screens/PaymentScreen.tsx
const validateSortCode = (sortCode: string): string | undefined => {
  if (!sortCode) return 'Sort code is required';
  if (!/^\d{6}$/.test(numbersOnly)) return 'Sort code must be exactly 6 digits';
  return undefined; };
const handleSortCodeChange = (text: string) => {
  const formatted = formatSortCode(text);
  setRecipient({ ...recipient, sortCode: formatted });
  validateField('sortCode', formatted);
```

Figure 3.10: Payment Screen Initiation

Figure 3.11 shows the payment screen, with the real-time validation checks, and well labelled fields.

Figure 3.11: Payment Screen

Spending Categorisation

Sample code in figure 3.12 shows how the app sends transactions to the AI, after performing the keywords check to group the different various transactions into categories. The AI comes into play, when any of the transactions do not fit any of the keywords or the user-defined categorisation. The reason for using this hybrid method was to reduce API cost and improve

response time. During various testing, instead of passing between 65 – 75 transactions through the AI, the keyword matching reduced that to 10 – 12, allowing for more processing and reducing the cost.

```
if (finalUnclearDescriptions.length > 0) {
  try {
    console.log('\n🤖 Sending to AI for categorization:');
    finalUnclearDescriptions.forEach((desc, index) => {
      console.log(`  ${index + 1}. "${desc}"`);
    });
  }
}
```

Figure 3.12: Spending Categorisation

Figure 3.13 shows the spending categorisation after it has been sorted by the keyword and AI.

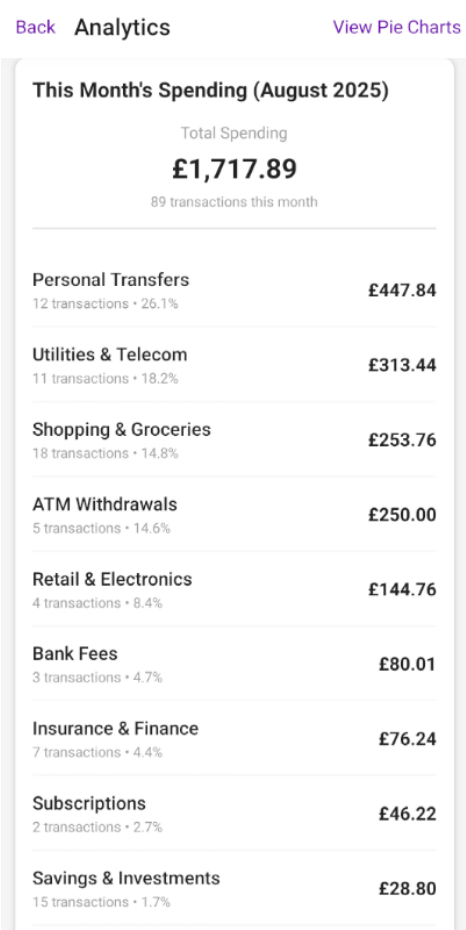


Figure 3.13: Spending Categorisation Screen

Figure 3.14 shows the category management screen, that shows the user defined rules, and the AI sorted categories saved in the firebase database.

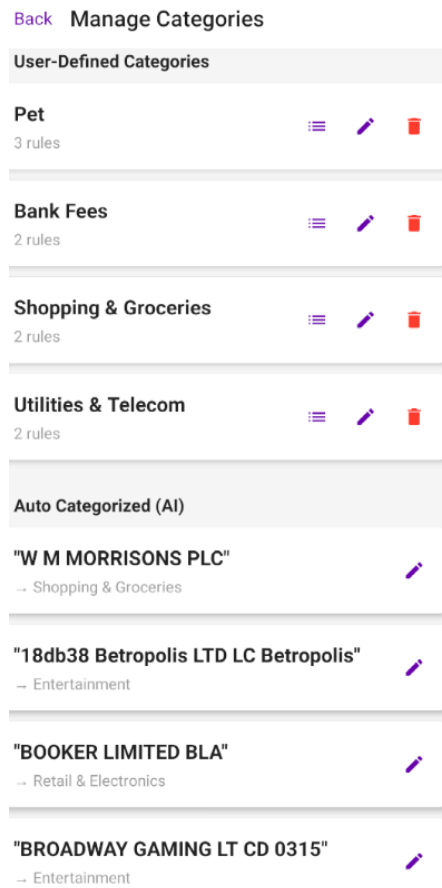


Figure 3.14: Category Management Screen

3.4.4 Security Implementation

Security considerations were part of every development stage of the application design because of the sensitive nature of financial data. To meet the industry standard required for a financial application, a layered security approach was used [6].

Authentication and data protection were part of the foundation of this project, and the Firebase Authentication was configured to work with the encryptionService.ts to encrypt the data transfer between the firebase database and the application is encrypted [7]. Access to log into the app is configured to support both email/password authentication and biometric verification, this depending on the device capability.

Secure storage through the secureStorage.ts used keychain services integrated with the service to protect sensitive authentication tokens and user credentials. Keychain was selected over alternatives like AsyncStorage or custom encryption as these do not have hardware backed security.

Data storage in the app follows a top to bottom system: authentication tokens use the hardware-backed Keychain storage, user data and bank connection details are in encrypted format in Firebase Firestore, and cached data expire automatically via set retention policies in the AsyncStorage. The reason for this top to bottom approach is that it creates a security classifica-

tion where the most sensitive data like tokens and credentials receives the strongest protection. While less sensitive data like UI cache or batch results are stored in temporary cache.

3.4.5 Testing and Quality Assurance

Testing was conducted throughout the development process to make sure each component functioned correctly. The final testing was conducted on 6 different Android devices with different OS versions to verify compatibility. Bugs and challenges were logged with documented steps to reproduce the solutions applied. The choice of using 5 users is based on a research by Nielsen, showing that 5 users can identify 85% of usage issues in testing [9].

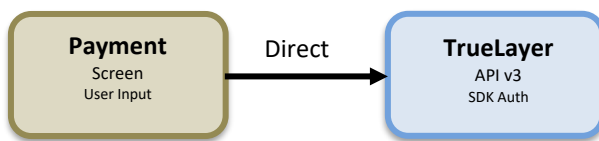
3.5 Challenges and Resolutions

App development hardly goes according to plan, and this project was no exception. There were many challenges encountered during the development process that required creative problem-solving and, in some cases, complete architectural pivots. Some of challenges have already been covered above, but below are the other development issues and their solutions faced in the coding of this project.

1. **SDK compatibility issues:** Before the decision to switch to WebView-based integration, the build issues encountered came up, when integrating the plugin into the project. Many changes were made to the `android/build.gradle`, `android/app/build.gradle` and `mainapplication.kt/java`, to get the build to run without flagging compatibility issues. After failing to resolve these issues, a switch to WebView integration was made. There is as of this writing no documentation found that detailed the needed settings, React Native version and `package.json` dependency list that shows how to link the SDK to a React Native project.
2. **Java memory limitations:** Regardless of the React Native framework, "Java heap space" errors occurred during the build process, causing build crashes that required a system reboot. A permanent solution was to increase JVM memory allocation in `gradle.properties` to 2048MB with additional metadata space.
3. **Slow build time:** The firebase plugin/dependencies along with the Flipper debugging tool caused the build time to be 12 minutes, marking it hard to carry out quick testing and resolution of issues. Since firebase is part of this app functionality, disabling Flipper in `build.gradle` and renaming `ReactNativeFlipper.java` to `old_ReactNativeFlipper.java.bak` reduced build time by half.
4. **Console error management:** The cleaned UI error stopped the app navigation flow, the app would not navigate from the `BankAuthScreen.tsx` to the `DashboardScreen.tsx`, because the UI error pops up on the mobile screen. The solution was to disable all errors from popping up on the UI and make them show only in the logs.

3.5.1 Payment Authorisation Challenge

SDK Approach (Intended)



WebView Approach (Implemented)

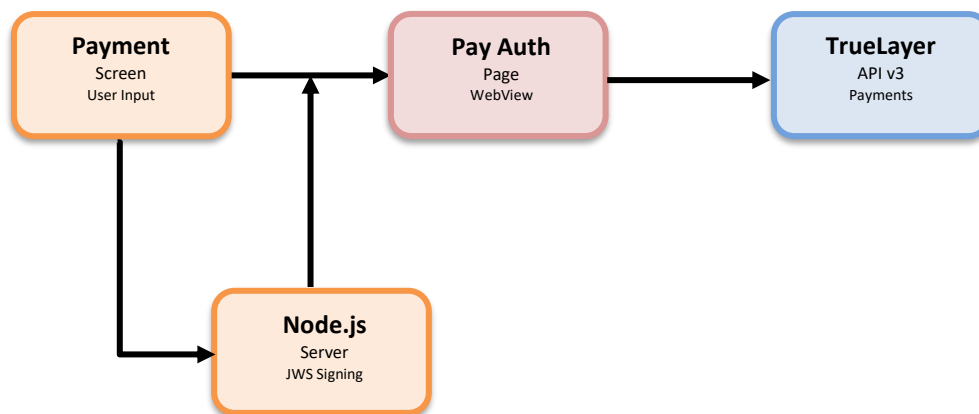


Figure 3.15: Payment Flow

Figure 3.15 shows the difference in the payment flow when comparing the SDK approach to the WebView approach.

Native SDK has higher performance in resource and energy consumption, while using WebView give developers the advantage of better flexibility and fewer environmental dependency issues [24]. However, these advantages are context based and does not apply in all application development.

One notable disadvantage of WebView is that they are susceptible to broader attack surfaces [25], while native SDK are more secure. Limited documentation exists on balancing both methods in banking applications, creating trial-and-error burdens for developers a challenge faced in this project.

The move from SDK to WebView had its own challenges, which included an additional screen to handle the WebView, which would have been handled by the SDK internally.

Since the project has pivoted to using WebView, the payment request has to be cryptographically signed with a TI-signature header before it can be accepted and processed by Truelayer. TrueLayer's payment endpoint (API v3) uses a specific JWS (JSON Web Signature) format, using Elliptic Curve Digital Signature Algorithm (ECDSA ES512) with P-521 curve and SHA-512 hashing (see section 2.3.2). The signing also needs an Idempotency-Key, which is used to make each request unique and not a repetition [10]. It is this signed hashed message that will be the TI-signature header required in the request.

Another alternate technical change implemented with security and simplicity being the main drive, was to use server-side signing (section 2.3.2). The advantage of this process is that the private key needed for this authorisation is not included in the app, and resides outside it, making the app more secure and following key management standards [11] [12].

3.5.2 Production Deployment and Device Testing

The server-side signing solution described in Section 3.5.1 was effective on the Android emulator because the server is hosted locally. This presents a challenge when you need to run a test on a physical device which is not connected to the computer hosting the Node.js.

Since this is not a production environment, a free cloud service, which can host the Node.js for server-side signing to enable testing and development was sorted, and Render was selected.

It addressed the challenge of signing, without a local server for both the physical device and the emulator.

3.5.3 Slow Loading of Spending Category and AI API cost

The spending categorisation described in 3.4.3 had two issues: it was slow, and it hit rate limit after 15 transaction categorisation requests because the project uses free Gemini AI API which only allows 15 requests/minute limit. The slowness was because it had to check each transaction against the user-defined rules in the Firebase database online and not locally on the phone.

The rate limit issue was because the system lacked any memory of what was previously categorised. Without this memory, each categorisation process would reprocess the same transactions along with new ones, preventing any progress. For example, if 16 transactions need to be sorted with AI, it would only categorise the first 15. If it runs again since there is no memory, it would still only categorise the same first 15.

The solution used was to save the AI categorisation into the Firebase under the user's identity, along with the user-defined rules, then download these to cache to make access fast.

3.6 Lessons Learned and Summary

The development process showed the practical challenges, and the technical considerations needed to meet project timelines. The research exposed the real-world integration challenges experienced when trying to integrate financial SDKs and the importance of having alternative technical strategies. The successful technical changes showed why flexibility is often more valuable than stubbornly sticking to original plans, during technical development.

The experience revealed that dependency management requires patience and careful planning when dealing with React Native projects as newer framework versions do not mean better integration when working with SDKs.

In summary, this chapter documents the successful development of a secure mobile banking application using React Native 0.72.6, Firebase Authentication, TrueLayer's Open Banking APIs, and Google Gemini AI.

4 Evaluation

4.1 Introduction

This chapter assesses the performance of the app against the research questions, this section evaluates how the application addressed the integration to TrueLayer APIs, the security implementation and the unified account display. This is done by examining achievements, challenges and adaptations required during development. The table 4.1 below gives an overview of two of the three research questions and status concerning this app and the chapter will further breakdown the table into sections covering user experience evaluation, technical performance assessment, and compliance verification.

Research Question	Achievement	Details
Q1: How effectively can TrueLayer's Open Banking APIs be integrated into a React Native application?	✅ Successfully Achieved	Integration functionality includes account aggregation, transaction history, payment initiation, encrypted storage, and AI-powered categorisation.
Q2: How can a unified financial dashboard improve users' financial management experience?	✅ Successfully Achieved	Positive user feedback: "easy to see all my accounts in one place", "Love how the dashboard shows total balance right away", "This is exactly what mobile banking should be like", "Best mobile banking experience I've used"

Table 4.1: Research Questions Achievement Overview

4.2 User Experience Evaluation

Evaluating the apps quantitative performance metrics, security assessment, user experience, compliance and performance measurement. These metrics were gathered during user assessments on the app's friendliness and features in the testing phase of the application.

The app was tested by five users through hands-on testing sessions and the reason for five users is based on a research by Nielsen, showing that 5 users can identify 85% of usage issues in testing [9].

The user feedback was analysed using thematic grouping and the responses were grouped into navigation, performance, security and overall experience. The positive and negative comments in each group were then compared to identify patterns in the user's experience.

4.2.1 Navigation and Usability Assessment

User testing showed positive navigation experiences, but some areas were said to be needing improvement. User's feedback showed that the app's navigation is familiar to that of other banking apps, and the navigation design shows general acceptance. The confusion around features like bank selection shows where there are gaps in communicating the app's limitation. Participants noted "Pretty easy overall, the dashboard shows everything clearly" and "Really intuitive actually. Coming from other banking apps this feels much more modern." The unified dashboard showed the consolidated view of multiple account information and allowed users to

easily access transaction history for each account. This received positive feedback, with participants noting "Love how the dashboard shows total balance right away,"

However, areas for improvement centred on the Connect Bank Screen of the app, as some participants experienced initial complexity, noting "Bit confusing at first but got used to it" and confusion about bank selection: "the connect bank screen was confusing. Lots of banks listed but they all seem to do the same thing?". These user concerns showed that there is a need for self-explanatory text in the app interface to guide users on what to do and what to expect. Since the banks were all connected to the TrueLayer Mock Bank, an onboarding guide or a coming soon below the other banks would have helped users to understand that the other links do not work.

4.2.2 Performance and Reliability Perception

The performance assessment confirms the technical limitations detailed in section 4.3.3, where initial dashboard load time length stems from TrueLayer Mock bank environment rather than the application itself. While users noted the 6.41-second dashboard load time was long, their positive feedback on subsequent load performance shows that the caching strategy worked. In a production environment, improvements to the API integration would resolve most these concerns.

User experience revealed mixed performance perceptions, the strongest praise came from the biometric authentication speed and dashboard responsiveness. However, performance concerns centred on loading times and stability issues. As one participant reported "Had some loading times when first opening the app, took maybe 4-5 seconds," while another noted "Loading can be slow sometimes. The payment confirmation screen took a while to update status." A third participant experienced an app crash during bank selection, subsequent investigation showed this was device specific and not an app related issue. The slow loading time for the initial connection to the dashboard would best be addressed by replacing generic loading messages with specific status update with estimated completion time. For things like the payment screen a loading bar or icon, to indicate progress can be implemented.

4.2.3 Security Perception and Trust

The security assessment reveals that users want a balance between security of their financial data and flexibility to improve their experience. The feedback details how the successful implementation of security features made the users trust the app's management of their data.

Regarding security, they were positive feedback from users indicating they trust the app's management of their data, some of the feedback are "Excellent security features. The session timeout is perfect, and biometric login is smooth" and "Top notch security, maybe too paranoid with the timeouts but better safe than sorry."

However, two participants had concerns about the session timeout aggressiveness, while one participant mentioned the strictness in password requirements when registering a new user. To address the session timeout concerns, a configurable timeout setting can be added to allow users to set their preferred timeout setting. But concerning the password requirements, these align with standard password requirements for security [8], to secure sensitive user and financial data.

4.3 Technical Performance Assessment

4.3.1 TrueLayer Open Banking API Integration Effectiveness

The application achieved its functionality coverage as detailed in Table 4.2 below, even though there were some adaptive flexibilities from the original implementation intent. The payment functionality was achieved by using WebView instead of TrueLayer's SDK as discussed in section 3.5.2.

	Functionality	Status
1	TrueLayer Open Banking API integration	Achieved
2	React Native cross-platform app	Achieved
3	PSD2-compliant authentication	Achieved
4	Biometric verification	Achieved
5	GDPR-compliant data management	Achieved
6	End-to-end encryption	Achieved
7	Secure storage for sensitive information	Achieved
8	Unified financial dashboard	Achieved
9	Account balances display	Achieved
10	Transaction history	Achieved
11	Spending categories with visualisation	Achieved
12	Firebase Authentication	Achieved
13	Mock financial data handling	Achieved
14	Bank account aggregation	Achieved
15	TrueLayer SDK integration (pivoted to WebView)	Achieved
16	AI-powered transaction categorisation	Achieved

Table 4.2: Functionality completion

The integration of TrueLayer's Open Banking APIs functioned adequately and after completion of the integration, account data retrieval reached a 99% success rate, with 1% failures due to token retrieval issues, on the device because of network failure, and not the apps implementation. While 99% is a good result, that 1% failure, even though it was not from application implementation, can be problematic because users expect 100% access to their financial data. To improve user experience and make navigation faster, the app was developed to reduce API calls, and a cache system was implemented with data currency set to 2-minute expiration. This balances data freshness with API call efficiency but means the user's dashboard view is not always real-time. Since the application is accessing Mock Bank data, there was slowness noticed during API calls, which could be as a result of lack of a dedicated server to handle the calls. The cache stores account balances, transaction history, and AI categorisation results in unencrypted AsyncStorage, but it is cleared when the user logs out or the session times-out to make sure that user data is wiped off the device once the user closes or is logged out of the app. Further improvement could be to reduce the cache expiration time, or in a production environment, having the server prepared to handle huge volumes of API calls.

While the application meets basic regulatory requirements for a development environment, production usage requires all financial data, locally or cloud stored to be encrypted [6]. Also, there are many more security hardening measures that will also need to be done to meet mobile security guidelines [22] and to fully meet GDPR and PSD2 compliance requirements [18]. This project shows how architectural changes can achieve functional objectives when primary integration methods become constraints to development.

4.3.2 Authentication Performance

The authentication system showed performance within the 2 seconds, login via email/password averaged 1.38 seconds and Biometric authentication averaged 1.81 seconds. However, the biometric implementation had compatibility challenges on one test user's device due to hardware limitations.

Biometric authentication was introduced to improve security and user experience, which are key factors in financial application development. The project used both email/password and

biometric authentication methods and this addresses the traditional tension between security and usability that Kruzikova et al. [5] identified in their analysis of European mobile banking applications. If the user's device does not support Biometric login, the application defaults to email/password login as an alternative.

While Biometric authentication provides convenience to users to have access to their app without the need of constantly typing passwords, the lack of it can lead to bad password practices. The need to constantly type password can make a user decide to use a singular password for all their apps, leading to password reuse, which is a security issue in the case the password gets leaked. Further improvement in a production environment will be to make sure a comprehensive testing is done on hardware compatibility before application launch.

4.3.3 Unified Dashboard Performance

The dashboard performance showed mixed results, app navigation after the dashboard has loaded averaged 1.83 seconds for general sections and 1.65 seconds for account switching. However, initial dashboard load time averaged 6.41 seconds, which exceeds the average load time of other banking applications tested.

This loading delay was identified by a user, and in future implementation if this is not addressed or expectations well managed this can cause user abandonment. For comparison other banking apps like Monzo (3.5 seconds) or Bank of Scotland (4.34 seconds) were able to achieve this load time about 2 seconds faster, this is based on user testing performed for this project. This extended time was because of data retrieval from TrueLayer APIs, Firebase database, account aggregation, and balance loading. All these occur during the first dashboard access. These delays are due to not having a dedicated server for this implementation, using TrueLayer Mock Bank and using the free version of Firebase. These can be improved in a production environment, with a dedicated server with better network throughput.

However, once the dashboard has loaded, subsequent navigation was quicker due to caching, as explained in section 4.3.1. Users were content with the performance of the navigation after the initial dashboard has loaded, as seen in section 4.2.1.

4.4 Compliance and Security Assessment

The application implements several of the needed security compliance features across GDPR, PSD2 and general banking standards, although there are some limitations, which will be discussed below.

The application's GDPR compliance includes user consent collection through TrueLayer's Web-View interface, and the application does data deletion during logout to wipe the cached memory as explained in section 4.3.1. However, the application stores this cached memory unencrypted which is a GDPR data protection gap. For this reason, the data is stored locally on the device and not transmitted in the unencrypted format to firebase database which the app uses as cloud memory. To get full GDPR data protection compliance, encryption is required at all points.

PSD2 Strong Customer Authentication requirements are met through dual authentication either by email/password or biometric verification. Also, the app communicates securely via the API communication with TrueLayer's infrastructure be it for account aggregation or payment authorisation as seen in section 3.5.2.

Other security enhancements implemented in this project include encrypted storage for authentication tokens using react-native-keychain, session timeout management, and Firebase's built-in security features for user authentication.

4.5 Evaluation Summary

The evaluation had mixed results across the three research questions, but the implementation success indicates that Open Banking integration in React Native applications is possible, techni-

cally. While authentication performance was good and users responded positively to the unified dashboard concept, the assessment revealed critical areas requiring improvement, like the initial dashboard load time, onboarding clarity, and production-level security and data encryption.

The technical adaptations used during development, especially the SDK to WebView, had no effect on the user, as feedback did not indicate any issue when using the payment module. User feedback also valued the unified dashboard view, of seeing all their accounts with access to their transaction history on the consolidated dashboard, while noting some aspect of improvements in the connect bank feature that leads to the dashboard interface.

While the project addressed basic banking fragmentation challenges through functional Open Banking integration, several compliance gaps and performance limitations show that additional development would be required for a production environment.

The project mock bank development environment may have met the basic functions and addressed the research questions adequately, but these results are a foundation for understanding implementation challenges and to indicate areas requiring further research and development. These findings provide initial evidence that mobile frameworks can deliver consolidated financial experience.

5 Conclusion

5.1 Introduction

This project developed a React Native 0.72.6 banking application that integrates TrueLayer's Open Banking platform, Firebase and Google Gemini AI to create a financial application that unifies users' bank accounts. The application consolidates an authenticated user's bank accounts, allows for the process payments, and provides AI-powered transaction categorisation in the mock bank environments. There were, however, development challenges that required alternative technical approaches that raised questions about Open Banking's practicality in the real world.

The banking application achieved its core objectives and user testing with five participants showed the benefits of the unified dashboard concept (section 4.2.1). Their positive feedback showed that users value consolidated financial tools, and that React Native as a development environment can create such an experience.

The app implemented several of the security requirements needed by GDPR and PSD2 [18] (section 4.4) and it implemented dual authentication (email/password and biometric). It encrypted the storage using react-native-keychain for sensitive tokens (section 3.4.4), secure API communication with TrueLayer's infrastructure, and session timeout management.

5.2 Comparison to Other Work

The development of this banking application using React Native is in line with studies on hybrid mobile frameworks and Open Banking integration, showing the development advantages. Similar to Rieger and Majchrzak [16], the project development timeline limitation shows the practical advantages and implementation challenges of cross-platform environment over native development.

In comparison with Kotios et al. [26], the implementation of a hybrid AI approach for transaction categorisation showed some implementation challenges not addressed in prior literature. The experience with SDK integration and practical challenges of hybrid AI approach are not always captured in existing studies. Most literature focuses on performance gains and trade-offs, and it rarely addresses dependency management, API rate limits or the need of alternative technical pivots, all of which are encountered here. Also, usability studies always focus on an ideal environment and not the practical limitations of server downtime, mock environments or processes that may affect application performance.

This project aligns with observed trends and may be used as a practical insight into the real-world constraints of developing mobile financial applications.

5.3 Limitations

The project faced constraints in development due to scope:

1. The TrueLayer mock environment does not represent production behaviour, limiting the ability to measure the project performance under high transaction loads.
2. Using a free third-party LLM for transaction categorisation without any practical testing to evaluate the performance of a third-party LLM vs in-house LLM. Also, there were limits on the amount of categorisation the third-party LLM could do per minute (section 3.5.3).
3. The server side signing host, Render's free service shuts down the server after 15 minutes of inactivity, so payment testing always requires the server to be restarted (section 3.5.2).

These limitations show that while the application met its objectives, the findings must be interpreted within the context of this project.

5.4 Critical Implementation Challenges and Practical Insights

Despite the success of a working mobile application, the development process showed that there is still more optimisation and development documentation to be done to achieve standardisation of Open Banking implementation.

The SDK compatibility issue (section 3.5.1) shows why more documentation is needed to aid developers on managing dependencies during the implementation of financial SDKs for mobile financial applications. Also, the performance limitations which occurred in the unified dashboard's initial load time was because of the Open Banking platform's authentication protocols, and data aggregation processes. This was not an application development issue and shows that the current Open Banking infrastructure may need optimising for better performance on mobile applications.

These challenges raise questions about how ready current industry standardisation efforts such as the Berlin Group's NextGenPSD2 framework [37] are, in comparison with the practical challenges faced by developers.

The project showed that any Open Banking implementation may require technical pivots to achieve its primary objectives, and management of expectations concerning account aggregation.

5.5 Industry Implications and Practical Insights

The project findings are practical evidence that Open Banking technical integration is achievable but with some caveat. There are hidden costs in development time, architectural complexity, and ongoing maintenance requirements, which will come into play during development. This challenges the idea that standardisation like the Berlin Group's NextGenPSD2 framework [37] simplifies third-party financial application development.

For developers considering similar projects, it is important to budget additional time for dependency management and possible SDK compatibility issues, as seen in this React Native financial application. Second, have prepared alternative implementation strategies, in case the primary approaches fail or run into compatibility issues. Third, prioritise performance optimisation from the start, because Open Banking's multiple API calls, authentication protocols and data aggregation may cause latency challenges.

The project also indicates that the current evaluation methodology used here may inadequately assess Open Banking's potential. To get a real picture of the impact longitudinal studies that test, financial behaviour, adoption, performance and not just usability testing will need to be conducted.

5.6 Future Work

Several areas in this application development require improvements and investigations for future scaling of the application.

For technical development, proper assessment requires scaling to a production-level environment with large datasets. The present evaluation was done using a Mock Bank in a sandbox environment. Scaling it to a production-level will give a better assessment of the application's performance under realistic conditions instead of relying on the evaluation based on mock environmental limitations. Also, implementing lazy loading techniques could improve the initial dashboard load time identified in section 4.3.3 as a user's experience challenge.

The AI service could be expanded beyond the current transaction categorisation functionality, to include budgeting tools and personalised financial planning, building on the existing AI infrastructure.

Security improvements like fraud detection mechanisms that would protect the user from falling victim to scams or unauthorised payments can be added to increase the user's confidence in the application's management of their finances. Concerning the server-side cryptography

approach detailed in section 3.5.1, further analysis can be done to see the application's performance when the cryptography is done from the client-side.

User experience improvements like onboarding tutorials that clarify features like bank selection processes will address the limitations raised by users during testing. Accessibility implementations for the visually impaired and multilingual support will make the application accessible to users with different needs and language requirements.

5.7 Conclusion

This project shows that there is a disconnect between banking regulatory theories and development realities.

The application's objectives of unified dashboard functionality, secure authentication, and AI-powered categorisation were achieved, but the development process exposed gaps in existing literature. The SDK and infrastructure dependencies challenges described in section 3.3 are not addressed in current theories.

Positive user feedback on the unified dashboard (section 4.2.1) shows that there is a real demand for consolidated financial tools. Also, the project's technical adaptations proved that technical flexibility could overcome implementation barriers, but at a cost to development time and architectural simplicity and these hidden costs may make Open Banking innovation inaccessible.

The gap between regulatory standardisation efforts and practical challenges reveals that realising Open Banking's potential requires systematic improvements in its documentation, evaluation methodologies, and regulatory guidance.

Future work must move beyond proof-of-concept demonstrations to longitudinal studies, testing financial behaviour, adoption patterns, and production performance to bridge the gap between Open Banking's regulatory promises and practical reality.

References

- [1] M. Zachariadis and P. Ozcan, *The API Economy and Digital Transformation in Financial Services: The Case of Open Banking*, SWIFT Inst. Working Paper No. 2016-001, Jun. 15, 2017. [Online]. Available: <http://dx.doi.org/10.2139/ssrn.2975199>. [Accessed: Aug. 10, 2025]
- [2] S. Alsaqqa, S. Sawalha, and H. Abdel-Nabi, "Agile Software Development: Methodologies and Trends," *International Journal of Interactive Mobile Technologies (IJIM)*, vol. 14, no. 11, pp. 246–270, 2020. [Online]. Available: <https://doi.org/10.3991/ijim.v14i11.13269>. [Accessed: Aug. 10, 2025]
- [3] V. Kaushik, K. Gupta, and D. Gupta, "React Native Application Development," *International Journal of Advanced Studies of Scientific Research*, vol. 4, no. 1, 2019. [Online]. Available: <https://ssrn.com/abstract=3330011>. [Accessed: Aug. 10, 2025]
- [4] J. Vines, P. Dunphy, M. Blythe, S. Lindsay, A. Monk, and P. Olivier, "The joy of cheques: trust, paper and eighty somethings," in *Proc. ACM Conf. Comput.- Supported Cooperative Work (CSCW)*, New York, NY, USA, 2012, pp. 147–156. [Online]. Available: <https://doi.org/10.1145/2145204.2145229>. [Accessed: Aug. 10, 2025]
- [5] A. Kruzikova, L. Knapova, D. Smahel, L. Dedkova, and V. Matyas, "Usable and secure? User perception of four authentication methods for mobile banking," *Computers & Security*, vol. 115, p. 102603, 2022. [Online]. Available: <https://doi.org/10.1016/j.cose.2022.102603>. [Accessed: Aug. 10, 2025]
- [6] Payment Card Industry Security Standards Council, *Payment Card Industry (PCI) Data Security Standard Requirements and Security Assessment Procedures*, Version 3.2.1, May 2018. [Online]. Available: https://www.pcisecuritystandards.org/documents/PCI_DSS_v3-2-1.pdf
- [7] Google LLC, *System and Organization Controls (SOC) 3 Report over the Google Firebase System*, Dec. 2024. [Online]. Available: https://firebase.google.com/static/downloads/gdpr/Dec2024_Google_Firebase_SOC_3_Report.pdf. [Accessed: Aug. 12, 2025].
- [8] National Institute of Standards and Technology, "Strength of Passwords—NIST Special Publication 800-63B: Digital Identity Guidelines." [Online]. Available: <https://pages.nist.gov/800-63-4/sp800-63b/passwords/>. [Accessed: Aug. 5, 2025].
- [9] J. Nielsen, "Why you only need to test with 5 users," *Nielsen Norman Group*, 2000. [Online]. Available: <https://www.nngroup.com/articles/why-you-only-need-to-test-with-5-users/>. [Accessed: Aug. 5, 2025].
- [10] TrueLayer. *Request signing v2*, GitHub, [Online]. Available: <https://github.com/TrueLayer/truelayer-signing/blob/main/request-signing-v2.md>. [Accessed: Aug. 5, 2025].
- [11] OWASP Foundation. *Key Management Cheat Sheet*, OWASP Cheat Sheet Series, [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Key_Management_Cheat_Sheet.html. [Accessed: Aug. 5, 2025].
- [12] D. Danilo, *ECDSA Signatures with Node.js and Swift*, DEV Community, Apr. 2021. [Online]. Available: https://dev.to/_danilo/ecdsa-signatures-with-node-js-and-swift-365j. [Accessed: Aug. 5, 2025].
- [13] H. Rifà-Pous and J. Herrera-Joancomartí, "Computational and energy costs of cryptographic algorithms on handheld devices," *Future Internet*, vol. 3, no. 1, pp. 31–48, Mar. 2011. [Online]. Available: <https://doi.org/10.3390/fi3010031>. [Accessed: Aug. 24, 2025].
- [14] D. You and M. Hu, "A comparative study of cross-platform mobile application development," in *Proc. 12th Annu. Conf. Comput. Inf. Technol. Res. Educ. New Zealand (CITRENZ)*, Wellington, New Zealand, Jul. 2021, pp. 1–8. [Online]. Availa-

- ble: https://www.researchgate.net/publication/357898491_A_Comparative_Study_of_Cross-platform_Mobile_Application_Development. [Accessed: Aug. 10, 2025].
- [15] B. Suri, S. Taneja, I. Bhanot, H. Sharma, and A. Raj, "Cross-platform empirical analysis of mobile application development frameworks: Kotlin, React Native and Flutter," in Proc. 4th Int. Conf. Inf. Manage. Mach. Intell. (ICIMMI), Jaipur, India, Dec. 2022, pp. 1-6. [Online]. Available: <https://dl.acm.org/doi/10.1145/3590837.3590897>. [Accessed: Aug. 12, 2025].
 - [16] C. Rieger and T. A. Majchrzak, "Towards the definitive evaluation framework for cross-platform app development approaches," *Journal of Systems and Software*, vol. 153, pp. 175–199, 2019. [Online]. Available: <https://doi.org/10.1016/j.jss.2019.04.001>. [Accessed: Aug. 10, 2025].
 - [17] L. Faulkner, "Beyond the five-user assumption: benefits of increased sample sizes in usability testing," *Behavior Research Methods, Instruments, & Computers*, vol. 35, no. 3, pp. 379–383, Aug. 2003. [Online]. Available: <https://doi.org/10.3758/BF03195514>. [Accessed: Aug. 12, 2025].
 - [18] European Parliament and Council, "Directive (EU) 2015/2366 on payment services in the internal market (PSD2)," *Official Journal of the European Union*, L 337, 2015.
 - [19] European Commission, A study on the application and impact of Directive (EU) 2015/2366 on Payment Services (PSD2), 2022. [Online]. Available: <https://www.ecri.eu/sites/default/files/a-study-on-the-application-and-impact-of-directive-ev0423061enn.pdf>. [Accessed: Aug. 12, 2025].
 - [20] European Banking Authority, "EBA publishes clarifications to the seventh set of issues raised by its industry working group on Application Programming Interfaces under the Payment Services Directive (PSD2)," Jul. 2022. [Online]. Available: <https://www.eba.europa.eu/publications-and-media/press-releases/eba-publishes-clarifications-seventh-set-issues-raised-its>. [Accessed: Aug. 22, 2025].
 - [21] M. Gounari, G. Stergiopoulos, K. Pipyros, and D. Gritzalis, "Harmonizing open banking in the European Union: an analysis of PSD2 compliance and interrelation with cybersecurity frameworks and standards," *Int. Cybersecurity Law Rev.*, vol. 5, pp. 79–120, Jan. 2024. [Online]. Available: <https://link.springer.com/article/10.1365/s43439-023-00108-8>. [Accessed: Aug. 12, 2025].
 - [22] OWASP Foundation, "Mobile Application Security Testing Guide (MASTG)," OWASP, 2023. [Online]. Available: <https://mas.owasp.org/MASTG/>. [Accessed: Aug. 12, 2025].
 - [23] NIST, "Recommendation for Key Management – Part 1: General," NIST Special Publication 800-57 Part 1 Rev. 5, May 2020. [Online]. Available: <https://csrc.nist.gov/publications/detail/sp/800-57-part-1/rev-5/final>. [Accessed: Aug. 12, 2025].
 - [24] R. Horn et al., "Native vs Web Apps: Comparing the Energy Consumption and Performance of Android Apps and their Web Counterparts," *arXiv preprint arXiv:2308.16734*, Aug. 2023. [Online]. Available: <https://arxiv.org/abs/2308.16734>. [Accessed: Aug. 12, 2025].
 - [25] S. R. Kandula, "WebView Security Best Practices," *Int. J. Comput. Trends Technol.*, vol. 72, no. 12, pp. 171–178, 2024. [Online]. Available: <https://www.ijcttjournal.org/Volume-72%20Issue-12/IJCTT-V72I12P121.pdf>. [Accessed: Aug. 12, 2025].
 - [26] K. Kotios, C. Karaikos, S. Ntalampiras, D. D. Vergados, and A. L. Symeonidis, "Deep learning enhancing banking services: A hybrid transaction classification and cash flow prediction approach," *Journal of Big Data*, vol. 9, no. 1, pp. 1–21, 2022. [Online]. Available: <https://doi.org/10.1186/s40537-022-00651-x>. [Accessed: Aug. 12, 2025].
 - [27] European Banking Authority, *EBA Guidelines on Outsourcing Arrangements*, EBA/GL/2019/02, Feb. 25, 2019. [Online]. Available: <https://www.eba.europa.eu/regulation-and-policy/internal-governance/guidelines-on-outsourcing-arrangements>. [Accessed: Aug. 14, 2025].

- [28] J. Hoffmann *et al.*, "Training compute-optimal large language models," *arXiv preprint arXiv:2203.15556*, Mar. 2022. [Online]. Available: <https://arxiv.org/abs/2203.15556>. [Accessed: Aug. 14, 2025].
- [29] L. Chen, D. Moody, K. Randall, A. Regenscheid, and A. Robinson, "Recommendations for Discrete Logarithm-Based Cryptography: Elliptic Curve Domain Parameters," NIST Special Publication 800-186, National Institute of Standards and Technology, Feb. 2023. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-186>. [Accessed: Aug. 5, 2025].
- [30] National Institute of Standards and Technology, Digital Signature Standard (DSS), FIPS Publication 186-5, Feb. 3, 2023. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.186-5>. [Accessed: Aug. 5, 2025].
- [31] National Institute of Standards and Technology, Secure Hash Standard (SHS), FIPS Publication 180-4, Aug. 4, 2015. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.180-4>. [Accessed: Aug. 5, 2025].
- [32] S. Samsi *et al.*, "From Words to Watts: Benchmarking the Energy Costs of Large Language Model Inference," *arXiv preprint arXiv:2310.03003*, Oct. 2023. [Online]. Available: <https://arxiv.org/abs/2310.03003>. [Accessed: Aug. 24, 2025].
- [33] National Institute of Standards and Technology, *Security and Privacy Controls for Information Systems and Organizations*, NIST SP 800-53 Rev. 5, Sep. 2020. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-53r5>. [Accessed: Aug. 14, 2025].
- [34] A. Borzunov *et al.*, "Distributed inference and fine-tuning of large language models over the Internet," *arXiv preprint arXiv:2312.08361*, Dec. 2023. [Online]. Available: <https://arxiv.org/abs/2312.08361>. [Accessed: Aug. 14, 2025].
- [35] Microsoft Azure, *Compliance Offerings*, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/azure/compliance/offerings/>. [Accessed: Aug. 14, 2025].
- [36] Y. Nie *et al.*, "A survey of large language models for financial applications: Progress, prospects and challenges," *arXiv preprint arXiv:2406.11903*, Jun. 2024. [Online]. Available: <https://arxiv.org/abs/2406.11903>. [Accessed: Aug. 14, 2025].
- [37] The Berlin Group, "PSD2 access to bank accounts - NextGenPSD2 framework implementation," The Berlin Group, 2024. [Online]. Available: <https://www.berlin-group.org/psd2-access-to-bank-accounts>. [Accessed: Aug. 14, 2025].
- [38] A. Amarasinghe, "What is 'the Berlin Group' and NextGenPSD2?," *Medium*, Sep. 11, 2022. [Online]. Available: <https://medium.com/@akilaamarasinghe/what-is-the-berlin-group-and-nextgenpsd2-7e7fafeb96e9>. [Accessed: Aug. 14, 2025].
- [39] European Banking Authority, "EBA's response to the call for advice on the review of PSD2," EBA/Op/2022/06, Jun. 23, 2022. [Online]. Available: [https://www.eba.europa.eu/sites/default/documents/files/document_library/Publication/Opinions/2022/Opinion%20od%20PSD2%20review%20\(EBA-Op-2022-06\)/1036016/EBA's%20response%20to%20the%20Call%20for%20advice%20on%20the%20review%20of%20PSD2.pdf](https://www.eba.europa.eu/sites/default/documents/files/document_library/Publication/Opinions/2022/Opinion%20od%20PSD2%20review%20(EBA-Op-2022-06)/1036016/EBA's%20response%20to%20the%20Call%20for%20advice%20on%20the%20review%20of%20PSD2.pdf). [Accessed: Aug. 14, 2025].
- [40] TrustBuilder, "From PSD2 to PSD3: What's changing in the future of payments in Europe," Apr. 16, 2025. [Online]. Available: <https://www.trustbuilder.com/en/psd2-psd3-directive-future-payments-europe/>. [Accessed: Aug. 14, 2025].
- [41] OWASP Foundation, "OWASP Mobile Top 10," OWASP, 2024. [Online]. Available: <https://owasp.org/www-project-mobile-top-10/>. [Accessed: Aug. 17, 2025].
- [42] T. H. Chiboora, L. Chacha, T. Byagutangaza, and A. Gueye, "Evaluating Mobile Banking Application Security Posture Using the OWASP's MASVS Framework," in *Proc. ACM Conference*, 2023. [Online]. Available: <https://dl.acm.org/doi/fullHtml/10.1145/3588001.3609367>. [Accessed: Aug. 17, 2025].

- [43] PwC Ireland, *Payment Services Directive 3 (PSD3): What You Need to Know*, PwC, 2025. [Online]. Available: <https://www.pwc.ie/industries/banking/insights/payment-services-directive-3.html>. [Accessed: Aug. 17, 2025].
- [44] Finexer, *PSD3: All You Need to Know in 2025*, Finexer, 2025. [Online]. Available: <https://blog.finexer.com/psd3-all-you-need-to-know-in-2025/>. [Accessed: Aug. 17, 2025].